Algorithms for Data Science
Exam 2026

Group A
(Solutions)

June 2026

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	10	15	15	15	20	15	20	15	15	10	15	15	10	180
Score:														

*The exam has 13 tasks worth **180 points** in total. Duration: 2 hours. No aids allowed.*

1. **Asymptotic notation.** Simplify each expression to the simplest possible Θ form. [10]

   (a) $\Theta\big(5n^3 + 100n^2 + \log n\big)$ [2]

   Answer: _____

   Solution: $\Theta(n^3)$. The term $5n^3$ dominates both $100n^2$ and $\log n$ for large n.

   (b) $\Theta\big(n \log n^4 + n^2/1000\big)$ [2]

   Answer: _____

   Solution: $\Theta(n^2)$. Note $\log n^4 = 4 \log n$, so the first term is $\Theta(n \log n)$. Since $n^2/1000 = \Theta(n^2)$ dominates $\Theta(n \log n)$, the result is $\Theta(n^2)$.

   (c) $\Theta\big(2^n + n^{1000}\big)$ [3]

   Answer: _____

   Solution: $\Theta(2^n)$. Exponential functions grow faster than any polynomial.

   (d) Give the time complexity of the following code in Θ notation: [3]

```
1 for i = 1 to n do
2     for j = i to n do
3         for k = 1 to 10 do
4             print(i + j + k)
```

   Answer: _____

   Solution: $\Theta(n^2)$. The innermost loop runs exactly 10 times (constant). The double outer loop runs $\sum_{i=1}^{n}(n - i + 1) = n(n+1)/2$ times, which is $\Theta(n^2)$.

2. **Recurrences.** [15]

(a) Consider the following pseudocode for AlgoB: [5]

```
1  function AlgoB(n)
2      if n ≤ 1 then
3          return
4      i ← 1
5      while i ≤ n − 3 do
6          AlgoB(⌊n/6⌋)
7          i += 1
8      for j = 1 to 9 do
9          AlgoB(⌊n/6⌋)
10     for k = 1 to n^5 do
11         doWork()
```

Write the recurrence $T(n)$ for the worst-case running time of AlgoB(n).

Solution: The **while** loop starts at $i = 1$ and increments until $i > n - 3$, executing for $i = 1, 2, \ldots, n-3$: exactly $\mathbf{n - 3}$ recursive calls. The **for** loop makes exactly **9** recursive calls. Total: $(n-3) + 9 = \mathbf{n + 6}$ recursive calls.

$$T(n) = (n+6)\,T\big(\lfloor \tfrac{n}{6} \rfloor\big) + \Theta(n^5).$$

Note: the branching factor $n+6$ depends on n, so the Master Theorem does not apply directly.

(b) Solve the following recurrence using the **Master Theorem**. Show which case applies and explain. [5]

$$T(n) = 8\,T\big(\tfrac{n}{2}\big) + \Theta(n^2)$$

Solution: $a = 8$, $b = 2$, $f(n) = \Theta(n^2)$.
$c = \log_b a = \log_2 8 = 3$.
Since $f(n) = \Theta(n^2)$ and $c = 3 > 2$, Case A of the Master Theorem applies.
Result: $T(n) = \Theta(n^c) = \Theta(n^3)$.

(c) Solve the following recurrence using the **Master Theorem**. Show which case applies and explain. [5]

$$T(n) = 4\,T\big(\tfrac{n}{2}\big) + \Theta(n^2)$$

Solution: $a = 4$, $b = 2$, $f(n) = \Theta(n^2)$.
$c = \log_2 4 = 2$.
Since $f(n) = \Theta(n^2) = \Theta(n^c)$, Case B applies.
Result: $T(n) = \Theta(n^2 \log n)$.

3. **Sorting algorithms.** [15]

(a) You have an array of $n = 10^6$ positive integers where each value satisfies $1 \leq A[i] \leq n$. Which sorting algorithm would you use to sort this array most efficiently? Justify your choice and state its time complexity. [5]

> **Solution: Counting Sort**. Since all values lie in $[1, n]$, we allocate a counting array of size n (same as the input size). Time complexity: $\Theta(n + k) = \Theta(n + n) = \Theta(n)$. This is optimal and beats any comparison-based sort, which requires $\Omega(n \log n)$.

(b) Consider the following array: $A = \langle 5, 2, 8, 3, 9, 1, 7, 4 \rangle$. [5]
Show the complete execution of **MergeSort**: the recursive splitting into sub-arrays and each merge step. Write the state of each sub-array at every level.

> **Solution: Split phase**:
> Level 0: $\langle 5, 2, 8, 3, 9, 1, 7, 4 \rangle$
> Level 1: $\langle 5, 2, 8, 3 \rangle \quad | \quad \langle 9, 1, 7, 4 \rangle$
> Level 2: $\langle 5, 2 \rangle \mid \langle 8, 3 \rangle \quad | \quad \langle 9, 1 \rangle \mid \langle 7, 4 \rangle$
> Level 3: $\langle 5 \rangle \langle 2 \rangle \mid \langle 8 \rangle \langle 3 \rangle \quad | \quad \langle 9 \rangle \langle 1 \rangle \mid \langle 7 \rangle \langle 4 \rangle$
> **Merge phase**:
> Level 2: $\langle 2, 5 \rangle \mid \langle 3, 8 \rangle \quad | \quad \langle 1, 9 \rangle \mid \langle 4, 7 \rangle$
> Level 1: $\langle 2, 3, 5, 8 \rangle \quad | \quad \langle 1, 4, 7, 9 \rangle$
> Level 0: $\langle 1, 2, 3, 4, 5, 7, 8, 9 \rangle$

(c) MergeSort runs in $\Theta(n \log n)$ in the worst case. Does this mean it is the best possible comparison-based sorting algorithm? Argue your answer using the **decision-tree lower bound**. [5]

> **Solution: Yes**, $\Theta(n \log n)$ is optimal for comparison-based sorting. Any comparison-based algorithm can be represented as a decision tree where each internal node is a comparison and each leaf is a distinct permutation of the input. To sort correctly, every permutation of n elements must be reachable, so the tree has at least $n!$ leaves. The height of any binary tree with $\geq n!$ leaves is $\geq \log_2(n!) = \Theta(n \log n)$ (by Stirling's approximation). Hence any comparison-based algorithm requires $\Omega(n \log n)$ comparisons in the worst case, and MergeSort achieves this bound.

4. **Dynamic programming: stair climbing.** [15]

You are climbing a staircase with n steps. Each move you may climb either **1** or **2** steps. Let $dp[n]$ be the number of distinct ways to reach step n (starting at step 0).

(a) The recurrence is $dp[n] = dp[n-1] + dp[n-2]$ for $n \geq 2$. State the two base cases $dp[0]$ and $dp[1]$, and briefly explain why the recurrence holds. [3]

Solution: $dp[0] = 1$ (one way to stay at the ground without moving), $dp[1] = 1$ (only one step of size 1).
The recurrence holds because the last move to reach step n was either a single step (contributing $dp[n-1]$ ways) or a double step (contributing $dp[n-2]$ ways); these cases are disjoint.

(b) Using the base cases and the recurrence, fill in the DP table for $n = 0$ to 8. [7]

n	0	1	2	3	4	5	6	7	8
$dp[n]$									

Solution: $dp[0] = 1$, $dp[1] = 1$, $dp[2] = 2$, $dp[3] = 3$, $dp[4] = 5$, $dp[5] = 8$, $dp[6] = 13$, $dp[7] = 21$, $dp[8] = 34$.

(c) How many distinct ways are there to climb 8 steps? [3]

Answer: _____

Solution: $dp[8] = \mathbf{34}$.

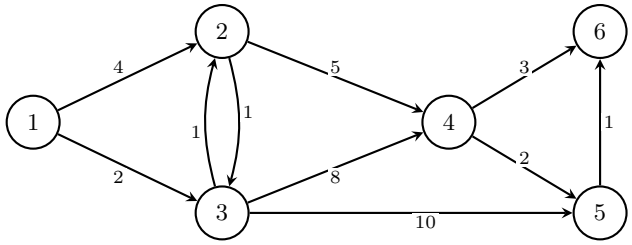
(d) What is the time complexity of computing $dp[0 \ldots n]$ using the recurrence? [2]

Answer: _____

Solution: $\Theta(n)$. Each entry is computed exactly once in a single left-to-right pass.

5. **Graph algorithms.** [20]

Consider the weighted directed graph below with vertices $\{1, 2, 3, 4, 5, 6\}$ and the following edges: $1 \to 2$ (4), $1 \to 3$ (2), $2 \to 3$ (1), $2 \to 4$ (5), $3 \to 2$ (1), $3 \to 4$ (8), $3 \to 5$ (10), $4 \to 5$ (2), $4 \to 6$ (3), $5 \to 6$ (1).



(a) Run **Dijkstra's algorithm** starting from vertex 1. Fill in the table below showing, for each step, which vertex is *extracted* and the updated *distance* values. [8]

Step	Extracted	$d[1]$	$d[2]$	$d[3]$	$d[4]$	$d[5]$	$d[6]$
0 (init)	—	0	∞	∞	∞	∞	∞
1							
2							
3							
4							
5							
6							

Solution: **Step 1**: Extract 1 ($d = 0$). Update: $d[2] = 4$, $d[3] = 2$.
Step 2: Extract 3 ($d = 2$). Update: $d[2] = \min(4, 3) = 3$, $d[4] = 10$, $d[5] = 12$.
Step 3: Extract 2 ($d = 3$). Update: $d[4] = \min(10, 8) = 8$.
Step 4: Extract 4 ($d = 8$). Update: $d[5] = \min(12, 10) = 10$, $d[6] = 11$.
Step 5: Extract 5 ($d = 10$). Update: $d[6] = \min(11, 11) = 11$ (no change).
Step 6: Extract 6 ($d = 11$). No outgoing edges.
Final distances: $d[1] = 0$, $d[2] = 3$, $d[3] = 2$, $d[4] = 8$, $d[5] = 10$, $d[6] = 11$.

(b) What is the shortest path from vertex 1 to vertex 6? List the vertices and the total distance. [5]

Solution: Path: $1 \to 3 \to 2 \to 4 \to 6$, total distance $= 2 + 1 + 5 + 3 = 11$.

(c) What is the time complexity of Dijkstra's algorithm when implemented with a **binary heap**? Express it in terms of n (vertices) and m (edges). [4]

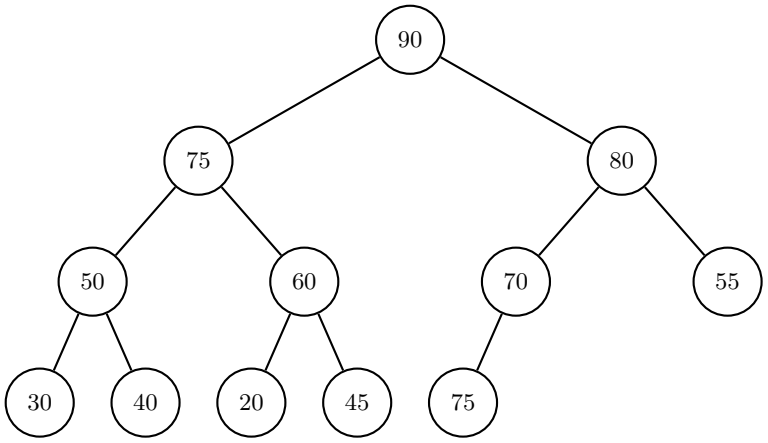
Solution: $\Theta((n + m) \log n)$.

(d) Run **BFS** (ignoring edge weights) from vertex 1. What hop-count distances does BFS report to each vertex? Give one example where BFS and Dijkstra disagree, and explain why. [3]

Solution: BFS hop distances: $d[1] = 0$, $d[2] = 1$, $d[3] = 1$, $d[4] = 2$, $d[5] = 2$, $d[6] = 3$.
Dijkstra gives $d[2] = 3$ via $1 \to 3 \to 2$ (weight $2 + 1$), while BFS gives $d[2] = 1$ (direct edge, 1 hop). BFS treats all edges as weight 1 and is only correct on unweighted graphs.

6. **Heaps and Binary Search Trees.** [15]

(a) The array $A = [90, 75, 80, 50, 60, 70, 55, 30, 40, 20, 45, 75]$ (indexed 1..12) is claimed to represent a **max-heap**. The corresponding complete binary tree is drawn below. [5]



(i) Identify the *one* index where the max-heap property is **violated** (a node is greater than its parent). (2 pts)

(ii) Fix the violation by changing *exactly one* value in the array. State which index you change and to what value. (3 pts)

Solution: (i) **Violation**: $A[12] = 75 > A[6] = 70$ (child exceeds parent). Index **12**.
(ii) **Fix**: Change $A[12]$ to any value ≤ 70, e.g. set $A[12] = 65$.

(b) Starting from the root key **15**, insert the keys $\langle 9, 22, 6, 12, 18, 25 \rangle$ (in this order) into a **Binary Search Tree**. Draw the resulting BST. [5]

Solution:

```
        15
       /  \
      9    22
     / \   / \
    6  12 18  25
```

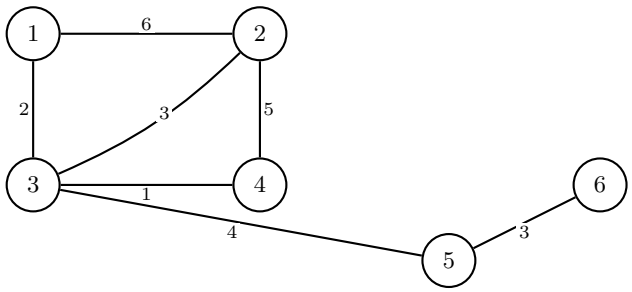
Insertions: 9 (left of 15), 22 (right of 15), 6 (left of 9), 12 (right of 9), 18 (left of 22), 25 (right of 22).

(c) After the insertions in part (b), check whether the resulting BST is also an **AVL tree**. Compute the height of each subtree and verify the balance condition at every node. If it is not an AVL tree, identify which node(s) violate the invariant. [5]

Solution: Heights (height of a leaf = 0): $h(6) = 0$, $h(12) = 0$, $h(9) = 1$, $h(18) = 0$, $h(25) = 0$, $h(22) = 1$, $h(15) = 2$.
Balance factors: 15: $|h(9) - h(22)| = |1 - 1| = 0$; 9: $|h(6) - h(12)| = |0 - 0| = 0$; 22: $|h(18) - h(25)| = |0 - 0| = 0$.
All balance factors ≤ 1, so this BST **is** an AVL tree.

7. **Minimum spanning tree — Kruskal's algorithm.** [20]

Consider the following weighted *undirected* graph with vertices $\{1, 2, 3, 4, 5, 6\}$:



(a) List all 7 edges sorted by weight in **non-decreasing** order. [3]

Solution: 3-4 (1), 1-3 (2), 2-3 (3), 5-6 (3), 3-5 (4), 2-4 (5), 1-2 (6).

(b) Run **Kruskal's algorithm**. For each edge (in sorted order), state whether it is **added** or **rejected**, and write the resulting connected components. [10]

Step	Edge	Add / Reject	Components after this step
1			
2			
3			
4			
5			
6			
7			

Solution:

Step	Edge	Action	Components
1	3-4 (1)	Add	$\{1\}, \{2\}, \{3, 4\}, \{5\}, \{6\}$
2	1-3 (2)	Add	$\{1, 3, 4\}, \{2\}, \{5\}, \{6\}$
3	2-3 (3)	Add	$\{1, 2, 3, 4\}, \{5\}, \{6\}$
4	5-6 (3)	Add	$\{1, 2, 3, 4\}, \{5, 6\}$
5	3-5 (4)	Add	$\{1, 2, 3, 4, 5, 6\}$ **(MST complete)**
6	2-4 (5)	Reject	2 and 4 already connected
7	1-2 (6)	Reject	1 and 2 already connected

(c) Draw the resulting MST (mark which edges belong to it). [3]

Solution: MST edges: 3-4 (1), 1-3 (2), 2-3 (3), 5-6 (3), 3-5 (4). Vertex 3 acts as a hub connecting all parts of the tree.

(d) What is the total weight of the MST? [2]

Answer: _____

Solution: $1 + 2 + 3 + 3 + 4 = \mathbf{13}$.

(e) What is the time complexity of Kruskal's algorithm in terms of n (vertices) and m (edges)? [2]

Solution: $\Theta(m \log m)$, dominated by sorting the edges.

8. **Complexity theory: P and NP.** [15]

(a) Briefly define the complexity classes **P** and **NP**. What does it mean for a problem to be **NP-complete**? [5]

> **Solution:** **P**: The class of decision problems solvable by a deterministic algorithm in polynomial time.
> **NP**: The class of decision problems for which a proposed solution (certificate) can be *verified* in polynomial time by a deterministic algorithm.
> **NP-complete**: A problem X is NP-complete if (1) $X \in$ NP, and (2) every problem in NP can be reduced to X in polynomial time (X is NP-hard). It is among the hardest problems in NP.

(b) For each problem below, state whether it is **in P** (an efficient polynomial-time algorithm is known) or **NP-complete** (no polynomial-time algorithm is known). Give a one-sentence justification. [6]

(i) Sorting n numbers.
(ii) Finding a shortest path in a weighted graph (all weights ≥ 0).
(iii) Deciding whether a graph has a Hamiltonian cycle (a cycle visiting every vertex exactly once).
(iv) Boolean Satisfiability (SAT): given a Boolean formula, is there a satisfying assignment?
(v) Finding a minimum spanning tree of a weighted graph.

> **Solution:** (i) **P** merge sort (or similar) runs in $O(n \log n)$.
> (ii) **P** Dijkstra's algorithm runs in $O((n+m)\log n)$ with a binary heap.
> (iii) **NP-complete** no polynomial-time algorithm is known; the best exact algorithms are exponential.
> (iv) **NP-complete** first NP-complete problem (Cook–Levin theorem, 1971).
> (v) **P** Kruskal's or Prim's algorithm runs in $O(m \log m)$.

(c) Mark **all** correct statements. [4]

- √ **If problem X is NP-complete and $X \in$ P, then P = NP.**
- ◯ Every NP-complete problem can be solved in polynomial time with parallel processors.
- √ **P $\subseteq$ NP.**
- √ **If problem X is NP-complete, then every problem in NP reduces to X in polynomial time.**
- ◯ All NP problems require exponential time to solve.
- √ **SAT is NP-complete.**

> **Solution:** Correct: 1st, 3rd, 4th, 6th. (2nd is false: parallelism does not generally collapse NP to P; 5th is false: P $\subseteq$ NP and all P problems are solvable in polynomial time.)

9. **Choosing the right data structure.** [15]

For each of the three scenarios below, choose the **most appropriate** data structure from the list and give a brief explanation. In your answer, state the relevant time and/or space complexity.

Available data structures:

1. Unsorted array
2. Sorted array
3. Hash table
4. Max-heap / Priority queue
5. BST (unbalanced)
6. AVL tree
7. Find-Union (Disjoint Set Union)
8. Queue

(a) A hospital's emergency room keeps track of n patients. New patients arrive and are assigned a priority score. The system must at any time be able to quickly retrieve **the patient with the highest priority** ($O(\log n)$) and add new patients ($O(\log n)$). Sorted output is not required. [5]

Most appropriate data structure: _____

Solution: Max-heap (Priority Queue). Supports `extractMax` in $O(\log n)$ and `insert` in $O(\log n)$. No need for sorted order or search by key, so a hash table or AVL tree would be overkill. Space: $\Theta(n)$.

(b) A social network has n users. Users form friend groups over time (merging groups when two users become friends). The system must answer queries: "are user A and user B in the same group?" in nearly $O(1)$ time, and merge two groups in nearly $O(1)$ time. [5]

Most appropriate data structure: _____

Solution: Find-Union (Disjoint Set Union). Supports `Union` and `Find` in $\Theta(\alpha(n))$ amortized time (nearly constant), where α is the inverse Ackermann function. It is specifically designed for dynamic connectivity queries. Space: $\Theta(n)$.

(c) A leaderboard for an online game stores n player scores. We need to: **insert** a new score ($O(\log n)$), **delete** a score ($O(\log n)$), **search** by player ID ($O(\log n)$), and **print all scores in sorted order** ($O(n)$). [5]

Most appropriate data structure: _____

Solution: AVL tree. Guarantees $O(\log n)$ insert, delete, and search in the worst case (unlike an unbalanced BST which can degrade to $O(n)$). In-order traversal gives sorted output in $O(n)$. A hash table would give $O(1)$ search but cannot provide sorted output. Space: $\Theta(n)$.

10. **Hash tables.** [10]

(a) Insert the keys $\langle 5,\ 12,\ 8,\ 3,\ 15,\ 10\rangle$ (in this order) into a hash table of size 7 using the hash function [4]

$$h(k) = k \bmod 7.$$

Collisions are resolved by **separate chaining** (each slot stores a linked list). Draw the resulting hash table, showing all chains.

Solution: $h(5) = 5,\ h(12) = 5,\ h(8) = 1,\ h(3) = 3,\ h(15) = 1,\ h(10) = 3.$

Slot	Chain
0	(empty)
1	$8 \to 15$
2	(empty)
3	$3 \to 10$
4	(empty)
5	$5 \to 12$
6	(empty)

(b) What is the **worst-case** time complexity for a search operation in a hash table with chaining? When does this worst case occur? [3]

Solution: Worst case: $\Theta(n)$, when all n keys hash to the same slot (one chain of length n). This occurs, for example, with adversarial input tailored to the hash function, or if all keys are congruent modulo the table size.

(c) Briefly compare a **hash table** and an **AVL tree** for the following operations. State the complexity and name one advantage of each data structure over the other. [3]

Operation	Hash table	AVL tree
Search		
Insert		
Delete		
Sorted output of all keys		

Solution:

Operation	Hash table	AVL tree
Search	$O(1)$ expected	$O(\log n)$
Insert	$O(1)$ amortized	$O(\log n)$
Delete	$O(1)$ expected	$O(\log n)$
Sorted output	$O(n \log n)$ (sort first)	$O(n)$ (in-order)

Hash table advantage: $O(1)$ average operations faster for simple insert/search/delete.
AVL tree advantage: supports sorted output in $O(n)$ and all operations in $O(\log n)$ *worst case*.

11. **Algorithm structure.** [15]

(a) The pseudocode below implements **BFS** (Breadth-First Search), but five expressions have been replaced by blanks. Fill in blanks **(a)**–**(e)** (shown as dashed lines) with the correct expression or value. [7]

(a) $dist[s] =$ _____

(b) condition of the **if**: _____

(c) $dist[w] =$ _____

(d) $parent[w] =$ _____

(e) argument to `add`: _____

```
1  dist[s] = (a)
2  parent[s] = NONE
3  q = new Queue
4  q.add(s)
5  while q not empty do
6      v = q.extractOldest()
7      for every edge (v, w) do
8          if (b) then
9              dist[w] = (c)
10             parent[w] = (d)
11             q.add((e))
```

Solution: (a) 0 (b) $dist[w]$ not yet computed (or $dist[w] = \infty$) (c) $dist[v] + 1$ (d) v (e) w

(b) The seven actions below describe **Kruskal's algorithm**, but they are in **scrambled order**. Assign each action a number from 1 to 7 indicating when it is executed (1 = first, 7 = last). [8]

(A) If FIND(u) $\neq$ FIND(v): add edge (u, v) to the MST and call UNION(u, v).

(B) Return the MST.

(C) Sort all edges by weight in non-decreasing order.

(D) Otherwise (both endpoints in the same component): discard this edge.

(E) For each edge (u, v, w) in sorted order (repeat until MST has $n - 1$ edges):

(F) Initialize MST as an empty list.

(G) Initialize a Find-Union structure with n singleton components (one per vertex).

Write the correct sequence here: (A) _____ (B) _____ (C) _____ (D) _____ (E) _____ (F) _____ (G) _____

Solution: The correct ordering: **F**(1), **C**(2), **G**(3), **E**(4), **A**(5), **D**(6), **B**(7).
Explanation: first initialize (F, G), then sort (C), then loop (E), inside the loop check each edge and either add (A) or discard (D), and finally return (B).

12. **Gradient descent and optimization.** [15]

(a) In supervised machine learning, we minimize a **loss function** $L(\theta)$ over parameters θ. [4]

(i) Write the **gradient descent update rule**. Define all symbols. (2 pts)

(ii) Explain intuitively why we move in the direction of the *negative* gradient. (2 pts)

Solution: (i) $\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$, where θ_t are the current parameters, $\eta > 0$ is the *learning rate*, and $\nabla L(\theta_t)$ is the gradient of the loss at θ_t.
(ii) The gradient $\nabla L(\theta)$ points in the direction of *steepest increase* of L. To *decrease* the loss we move in the opposite direction. Taking a small step in $-\nabla L$ guarantees (for small enough η) that the loss decreases locally.

(b) What happens if the learning rate η is: [3]

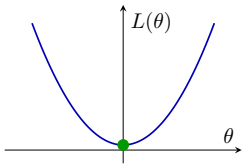
(i) Too small? (1.5 pts)

(ii) Too large? (1.5 pts)

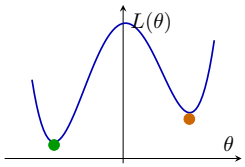
Solution: (i) **Too small**: Training progresses very slowly; many iterations are needed to approach a minimum.
(ii) **Too large**: The update overshoots the minimum; the loss may oscillate or diverge.

(c) Consider the two loss landscapes below. [4]

Landscape A



Landscape B



(i) Which landscape is **convex**? State one property that distinguishes a convex function. (2 pts)

(ii) For which landscape does gradient descent *always* find the **global** minimum, regardless of starting point? Explain why. (2 pts)

Solution: (i) **Landscape A** is convex. A function is convex if its graph lies at or below the chord connecting any two of its points; equivalently, it has no local minima other than the global minimum.
(ii) Gradient descent always converges to the global minimum on **Landscape A** (convex). On a convex function every local minimum is also the global minimum, so any descent path leads there. On Landscape B (non-convex) gradient descent may stop at the *local* minimum (orange dot) instead of the global one (green dot).

(d) Consider minimizing $L(w) = (w-3)^2$ using gradient descent with learning rate $\eta = 0.5$, starting at $w_0 = 0$. [4]

(i) Compute w_1 and w_2 (two gradient steps). (2 pts)

(ii) What is the minimum of $L(w)$? Does gradient descent converge to it here? (2 pts)

Solution: $\nabla L(w) = 2(w-3)$.
$w_1 = 0 - 0.5 \cdot 2(0-3) = 0 + 3 = 3$.
$w_2 = 3 - 0.5 \cdot 2(3-3) = 3$.
The minimum is at $w^* = 3$ with $L(3) = 0$. Yes, gradient descent reaches it in one step ($\eta \cdot 2 = 1$ equals the reciprocal of the curvature for this quadratic).

13. **Algorithm design: heat wave.** [10]

*A meteorological station records daily maximum temperatures $T[1..n]$ over n consecutive days. A **heat wave** is the longest consecutive sequence of days where each day is strictly warmer than the previous one.*

The pseudocode below is a *skeleton* of the HeatWave algorithm that returns the starting index and length of the longest strictly increasing run. Five lines have been replaced by blanks **(a)**–**(e)**.

(a) initial value of *bestLen* and *currLen*: _____

(b) condition to extend the current run: _____

(c) variable to update when a new best is found: _____

(d) variable to update when a new best is found: _____

(e) reset *currStart* to: _____

```
1  function HeatWave(T[1..n])
2      bestStart ← 1
3      bestLen ← (a)
4      currStart ← 1
5      currLen ← (a)
6      for i ← 2 to n do
7          if (b) then
8              currLen ← currLen + 1
9              if currLen > bestLen then
10                 (c) ← currStart
11                 (d) ← currLen
12         else
13             currStart ← (e)
14             currLen ← 1
15     return bestStart, bestLen
```

(a) Fill in blanks **(a)**–**(e)**. [5]

> **Solution:** (a) 1 (b) $T[i] > T[i-1]$ (c) *bestStart* (d) *bestLen* (e) i

(b) State the **time** and **space** complexity and briefly justify both. [2]

> **Solution:** **Time**: $\Theta(n)$ one pass, $O(1)$ work per element. **Space**: $\Theta(1)$ four variables only.

(c) Run the completed algorithm on $T = \langle\, 22,\ 19,\ 20,\ 23,\ 21,\ 24,\ 27,\ 29 \,\rangle$. Show your work (list the runs found) and state the starting index and length of the longest heat wave. [3]

> **Solution:** Runs: $\langle 22 \rangle$ (len 1); $\langle 19, 20, 23 \rangle$ (len 3, start 2); $\langle 21, 24, 27, 29 \rangle$ (len 4, start 5).
> **Answer**: start = **5**, length = **4**.